འབྲུག་རྒྱལ་འཛིན་གཙུག་ལག་སློབ་སྡེ།།

College of Science and Technology
Rinchending: Bhutan

CTE202
Data Structures and Algorithms
(SS2025)

Laboratory 9 Report

Submitted By;
Student Name: Ugyen Lhendup
Enrolment No.: 02230116
Programme: 3 ECE
Date: 22/04/2026

RUB Wheel of Academic Law: Academic Dishonesty

Section H2 of the Royal University of Bhutan's *Wheel of Academic Law* provides the following definition of academic dishonesty:

Academic dishonesty may be defined as any attempt by a student to gain an unfair advantage in any assessment. It may be demonstrated by one of the following:

1. **Collusion:** the representation of a piece of unauthorized group work as the work of a single candidate.
2. **Commissioning:** submitting an assignment done by another person as the student's own work.
3. **Duplication**: the inclusion in coursework of material identical or substantially similar to material which has already been submitted for any other assessment within the University.
4. **False declaration**: making a false declaration in order to receive special consideration by an Examination Board or to obtain extensions to deadlines or exemption from work.
5. **Falsification of data**: presentation of data in laboratory reports, projects, etc., based on work purported to have been carried out by the student, which has been invented, altered or copied by the student.
6. **Plagiarism**: the unacknowledged use of another's work as if it were one's own.

Examples are:

- verbatim copying of another's work without acknowledgement.
- paraphrasing of another's work by simply changing a few words or altering the order of presentation, without acknowledgement.
- ideas or intellectual data in any form presented as one's own without acknowledging the source(s).
- making significant use of unattributed digital images such as graphs, tables, photographs, etc. taken from test books, articles, films, plays, handouts, internet, or any other source, whether published or unpublished.
- submission of a piece of work which has previously been assessed for a different award or module or at a different institution as if it were new work.
- use of any material without prior permission of copyright from appropriate authority or owner of the materials used".

Topic: Implement Indexed Search and Selection Sort Algorithm

Aim: To implement Indexed Search and Selection Sort algorithms using Python.

Background:

Searching and sorting are fundamental operations in data structures and algorithms. Searching refers to the process of determining whether a specific element exists within a dataset. In this lab, students implement Indexed Search, a technique that improves search efficiency by using an additional index table to narrow down the search range. This method works by dividing a sorted list into smaller blocks, with the index table storing selected values and their corresponding positions. When searching for a key, the index table is first consulted to identify the likely range, after which a sequential search is performed only within that limited section.

Sorting, on the other hand, involves arranging data in either ascending or descending order. In this lab, students implement Selection Sort, a simple algorithm that repeatedly selects the smallest element from the unsorted portion of the list and places it in its correct position. The algorithm divides the list into sorted and unsorted sections and requires $n - 1$ passes for n elements. Although Selection Sort has a time complexity of $O(n^2)$ in all cases, it performs at most one swap per pass and uses only constant extra memory, making it an in-place algorithm suitable for small datasets where minimizing swaps is important.

Code:

```
####################################################
# Selection Sort and Indexed Search
####################################################

# Task 1 and 2
def selection_sort(arr):
    n = len(arr)
    comparisons = 0
    swaps = 0

    print("Original list:", arr)

    for i in range(n - 1):
        min_index = i

        # Inner loop to find the minimum element
        for j in range(i + 1, n):
            comparisons += 1
```

```
            if arr[j] < arr[min_index]:
                min_index = j

        # Swap if needed
        if min_index != i:
            arr[i], arr[min_index] = arr[min_index], arr[i]
            swaps += 1

        print(f"Pass {i + 1}:", arr)

    print("Sorted list:", arr)
    print("Total comparisons:", comparisons)
    print("Total swaps:", swaps)

    return arr


# Task 3
def create_index_table(arr, block_size):
    index_table = []

    for i in range(0, len(arr), block_size):
        index_table.append((arr[i], i))

    print("\nIndex table created:")
    for value, index in index_table:
        print(f"{value} -> {index}")

    return index_table


# Task 4 & 5: Indexed Search
def indexed_search(arr, index_table, key):
    print(f"\nSearch key: {key}")

    imin = 0
    imax = len(arr) - 1

    # Step 1: Find range using index table
    for i in range(len(index_table)):
        if i == len(index_table) - 1 or key < index_table[i + 1][0]:
            imin = index_table[i][1]
            if i == len(index_table) - 1:
                imax = len(arr) - 1
            else:
                imax = index_table[i + 1][1] - 1

            print("Index range found:")
            if i < len(index_table) - 1:
```

```
                print(f"{index_table[i][0]} <= {key} < {index_table[i +
1][0]}")
            else:
                print(f"{index_table[i][0]} <= {key}")
            break

    # Step 2: Sequential search in range
    print(f"Searching from index {imin} to index {imax}:")

    for i in range(imin, imax + 1):
        print(f"Checking index {i}: {arr[i]}")
        if arr[i] == key:
            print(f"{key} found at index {i}")
            return i

    print(f"{key} not found")
    return -1

#Example usage
arr = [29, 10, 14, 37, 13]
sorted_arr = selection_sort(arr.copy())

arr2 = [10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65]
index_table = create_index_table(arr2, 3)

indexed_search(arr2, index_table, 45)
indexed_search(arr2, index_table, 43)
```

Expected Outcome:

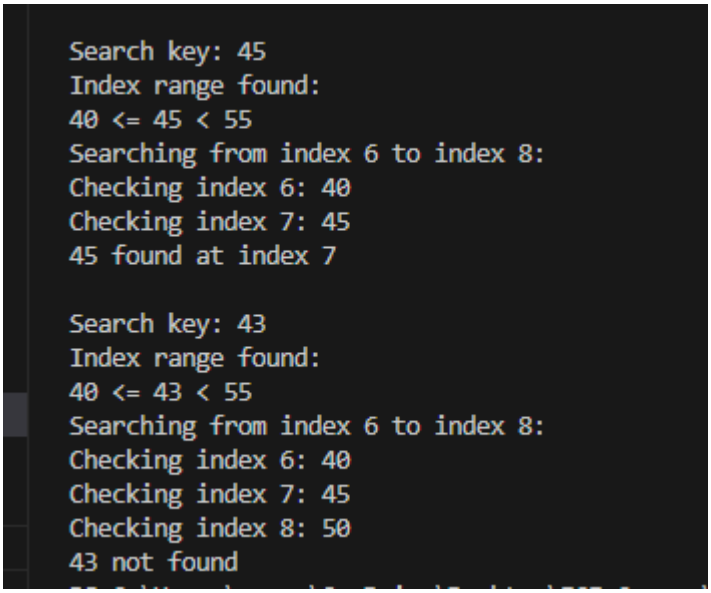
The program begins by defining a function called selection_sort, which takes a list of elements as input and sorts them in ascending order using the Selection Sort algorithm. It iterates through the list, and for each position, it assumes the current element is the smallest, then uses an inner loop to compare it with the remaining unsorted elements to find the actual minimum value. Once the smallest element is identified, it swaps it with the element at the current position if necessary. After each pass, the updated state of the list is printed, allowing users to observe how the list gradually becomes sorted. In addition to sorting, the program keeps track of the total number of comparisons made during the process as well as the number of swaps performed. At the end, it displays the fully sorted list along with the total comparisons and swaps, as shown below.

```
AB/02230116_LAB9.py"
Original list: [29, 10, 14, 37, 13]
Pass 1: [10, 29, 14, 37, 13]
Pass 2: [10, 13, 14, 37, 29]
Pass 3: [10, 13, 14, 37, 29]
Pass 4: [10, 13, 14, 29, 37]
Sorted list: [10, 13, 14, 29, 37]
Total comparisons: 10
Total swaps: 3
```

The program then defines a function called create_index_table, which constructs an index table from a sorted list. It takes two inputs: the sorted list and a specified block size. The function divides the list into smaller blocks based on the block size and selects the first element from each block. These selected elements, along with their corresponding positions in the original list, are stored as pairs in the index table. After building the table, the program prints each value and its index, clearly showing how the list has been partitioned into searchable segments as shown below.

```
Index table created:
10 -> 0
25 -> 3
40 -> 6
55 -> 9
```

Finally, the program defines a function called indexed_search, which performs an efficient search for a given key using the previously created index table. It first scans the index table to determine the range in which the key is most likely to be found by comparing the key with indexed values. Once the appropriate range is identified, the program sets the starting and ending indices and performs a sequential search only within that limited portion of the list. During the search, each checked element is printed to show the step-by-step process. If the key is found, the program outputs its index position; otherwise, it reports that the key is not found. The function is tested with both an existing key and a non-existing key, as shown below.



Github link: https://github.com/Lhenz4/CTE202-LAB